Fire Insurance Premium Prediction

Credit Agricole Assurances: Agricultural Multi-Risk Insurance

Abstract. *This report presents a complete data science pipeline for predicting the fire insurance premium (CHARGE) of agricultural firms insured by Credit Agricole Assurances. The dataset comprises 383,610 training observations with 373 features covering location, weather, farm type, insured property and prevention measures. We first conduct an unsupervised exploration using PCA and K-Means clustering, identifying five distinct risk segments. We then build a multiplicative two-stage supervised model: an XGBoost Poisson regressor for fire frequency (FREQ) and an XGBoost severity regressor for average claim cost (CM). The final premium is computed as CHARGE = FREQ x CM x ANNEE_ASSURANCE. Evaluated via 5-fold cross-validation, the model achieves a mean RMSE of 6,790, and a leaderboard RMSE of 5,603 - placing in the top tier of the course competition.*

1. Introduction

Credit Agricole Assurances is one of France's leading agricultural insurers, providing multi-risk coverage to farming enterprises. Accurate fire premium pricing is critical: under-pricing creates financial exposure, while overpricing reduces competitiveness. The premium for a given policyholder is defined as:

$$\text{CHARGE} = \text{FREQ} \times \text{CM} \times \text{ANNEE_ASSURANCE}$$

where FREQ is the expected number of fire events per year, CM is the expected average claim cost per event, and ANNEE_ASSURANCE is the number of years of insurance coverage. The modelling task therefore decomposes naturally into two sub-problems: frequency prediction and severity prediction. This report follows the full data science lifecycle - data exploration, unsupervised pattern discovery, supervised modelling, and deployment planning.

2. Data Description and Preprocessing

2.1 Dataset Overview

The dataset provided by Credit Agricole Assurances contains 383,610 training records and 95,852 test records, each described by 373 features and three target variables. Features span four broad categories: (i) geographic and meteorological variables, (ii) farm and insured property characteristics (building count, surface areas, capital values), (iii) prevention measures adopted by the policyholder, and (iv) administrative variables including policy duration. Some column names have been anonymised for privacy.

Attribute	Training Set	Test Set
Observations	383,610	95,852
Raw features	373	373
Numerical features	27	27
Binary features	11	11
Categorical features	332	332
Target: FREQ (mean)	0.0125	-
Target: CM (non-zero)	29,775 EUR	-
Target: CHARGE (mean)	186 EUR	-
FREQ = 0 (no fire)	99.25%	-

Table 1. Summary statistics of the dataset.

2.2 Target Variable Analysis

The target distributions reveal extreme sparsity: 99.25% of policyholders report zero fire events (FREQ = 0), and 99.39% have zero claim cost (CM = 0). Among the 2,352 non-zero CM observations, the distribution is highly right-skewed (skewness = 3.83), ranging from EUR 1 to EUR 500,000 with a median of EUR 1,787. This sparsity has direct implications for model design: standard regression would be dominated by zero-valued records. A small number of negative CM values exist (minimum = -EUR 5,751), likely representing insurance adjustments; these are clipped to zero as they cannot represent valid premiums.

2.3 Preprocessing Pipeline

Several data cleaning strategies were tested in order to improve the quality of the dataset before training the models. These included handling missing values using more complex transformations and additional preprocessing steps. However, the models trained on the cleaned dataset consistently performed worse than those trained on the dataset where missing values were imputed using simple statistical methods such as the mean, median, and mode.

One possible explanation is that the additional cleaning steps may have introduced noise into the dataset. When preprocessing procedures modify the original distribution of the data or replace values with estimates that are not representative of the underlying patterns, the resulting dataset can become less informative for the model. In contrast, simple imputation methods tend to preserve the overall distribution of the variables while minimally altering the structure of the data.

Another factor is that some machine learning models are relatively robust to missing data when it is handled with basic imputation techniques. Overly aggressive cleaning or transformation can remove useful variability or distort relationships between variables that the model relies on for prediction.

For example, a highly structured missing data pipeline was implemented following a decision flowchart classifying each column's missingness pattern as MCAR (Missing Completely At Random), MAR (Missing At Random), or MNAR (Missing Not At Random). The classification uses statistical tests (point-biserial correlation, Kruskal-Wallis) on a 10,000-row sample for efficiency. Treatment is then applied on the full dataset using the detected patterns. This complex method performed worse than simple, median, mode imputation.

Pattern	Detection Heuristic	Treatment Applied
MCAR <5%	No correlation with other vars, low rate	Listwise deletion
MCAR 5-15%	No correlation, moderate rate	Median / mode imputation
MCAR >15%	No correlation, high rate	Multiple imputation (sampled from observed)
MAR	Missingness correlated with observed vars	Multiple imputation / EM algorithm / Hot deck
MNAR	High rate, no MAR signal	Sensitivity analysis / Heckman / Pattern mixture

Table 2. Structured missing data pipeline.

Therefore, although more sophisticated cleaning approaches were explored, the final models were trained using mean, median, and mode imputation, as this approach produced more stable and better-performing results during validation. This highlights an important principle in data preprocessing: simpler methods can sometimes outperform more complex ones when they preserve the underlying structure of the data.

3. Unsupervised Learning: Dimensionality Reduction and Clustering

3.1 Principal Component Analysis

To explore structure in the numerical feature space, we applied Principal Component Analysis (PCA) to the 27 standardised numerical features. Standardisation (zero mean, unit variance) was performed prior to PCA to prevent features with large scales from dominating the decomposition. Thirteen components suffice to capture 90% of the total variance, indicating substantial redundancy and correlation among the raw numerical features. The first component alone accounts for approximately 29% of variance, reflecting the dominant role of property size and capital features. [Figure 1 - PCA Scree Plot - as generated by the notebook]

3.2 K-Means Clustering

K-Means clustering was applied to the 13-dimensional PCA representation to identify natural groupings of agricultural firms. The optimal number of clusters was selected using the elbow method, which plots within-cluster inertia against k. Although the curve is gradual - typical of high-dimensional data with overlapping distributions - a discernible change in slope occurs around k = 5, which was therefore selected. Clustering was performed with 10 random initialisations (n_init = 10) and a fixed random seed for reproducibility. [Figure 2 - Elbow Plot; Figure 3 - Cluster Scatter - as generated by the notebook]

3.3 Cluster Profiles and Business Insights

The cluster profiles below reveal meaningful risk stratification. Cluster 2 exhibits a mean fire frequency nearly eight times higher than Cluster 1, and a mean premium more than seven times larger, suggesting a segment of high-exposure farms. Clusters 0, 1 and 4 are low-risk segments, while Cluster 3 represents a medium-high risk group with elevated average claim costs.

Cluster	Mean FREQ	Mean CM (EUR)	Mean CHARGE (EUR)	Risk Level
0	0.009	112	115	Low
1	0.007	83	84	Lowest
2	0.059	606	610	High
3	0.025	474	485	Medium-High
4	0.015	112	112	Low-Medium

Table 3. Mean target values and qualitative risk level by cluster.

4. Supervised Learning: Frequency and Severity Modelling

4.1 Modelling Approach

Given the multiplicative definition of CHARGE, we adopt a two-stage modelling strategy consistent with classical actuarial frequency-severity decomposition. Two independent models are trained and their outputs combined at prediction time:

- **Stage 1 - Frequency model:** Stage 1 - Frequency model: Predicts FREQ, the expected number of fire events per policy year.
- **Stage 2 - Severity model:** Stage 2 - Severity model: Predicts CM, the expected average cost per fire event, conditional on at least one event occurring.

This decomposition separates fundamentally different statistical problems: frequency is a count process governed by exposure factors, while severity depends on property values and fire spread characteristics. Training the severity model only on records where FREQ > 0 avoids structural zeros contaminating the CM estimates.

The following actuarial design principles were embedded throughout the modelling pipeline. Non-negativity of all predictions is enforced via np.maximum(..., 0) applied to both frequency and severity outputs before multiplication. The policy duration (ANNEE_ASSURANCE) acts as an exposure offset, multiplied at the final charge computation stage to scale predictions appropriately per unit time. The Poisson and log-normal objectives handle the right-skewed distribution of claims without requiring manual outlier removal or transformation of the target beyond log1p. Finally, the severity model is trained exclusively on policies with at least one observed claim, avoiding the systematic downward bias that would result from including zero-cost policies in the severity estimation.

4.2 Algorithm Selection: XGBoost

Gradient Boosted Decision Trees, specifically XGBoost were selected as the modelling algorithm for both stages. This choice is motivated by several considerations:

- **Mixed feature types:** Mixed feature types: XGBoost natively handles numerical and ordinally encoded categorical features without requiring one-hot encoding, critical given 332 categorical columns.
- **Sparsity handling:** Sparsity handling: XGBoost's histogram-based tree splitting efficiently processes the high proportion of zero-valued binary features.
- **Non-linearity:** Non-linearity: Tree ensembles capture complex interactions without explicit feature engineering.
- **Scalability:** Scalability: GPU-accelerated histogram method (tree_method='hist') enables tractable training on 383,610 rows and ~370 features.
- **Interpretability:** Interpretability: Feature importance scores provide actionable business insights into key drivers of fire risk.

Alternative algorithms considered include linear regression (insufficient for non-linear relationships), Random Forest (higher memory footprint), and neural networks (difficult to interpret and prone to overfitting on sparse tabular targets). XGBoost offers the best balance of predictive accuracy, interpretability, and computational efficiency.

Computation time was an explicit consideration in algorithm selection. With 383,610 training rows and 370 features, methods such as Support Vector Regression or Gaussian Process Regression were ruled out due to their $O(n^2)$ or $O(n^3)$ complexity. Random Forest, while parallelisable, requires storing all trees in memory simultaneously, which proved memory-intensive at this scale. XGBoost's histogram-based gradient boosting (tree_method='hist') with GPU acceleration completes a full 5-fold cross-validation in approximately 15–20 minutes, making iterative experimentation tractable within the project timeline

4.3 Model Comparison

To satisfy the requirement of comparing multiple model types, three candidate approaches were evaluated for the frequency prediction stage, using a shared XGBoost severity model to isolate the effect of the frequency model choice. A Ridge Regression

baseline, a Random Forest (200 trees, max_depth=10, trained on 20% row subsamples for computational feasibility), and XGBoost were compared using 3-fold cross-validation on the full CHARGE target. The results are presented in Table X below.

Model	CV RMSE (€)	Std
Ridge Regression	6,799.7	±86.9
Random Forest	6,800.5	±86.4
XGBoost	**6,799.0**	**±87.3**

Notably, all three models perform comparably on this dataset. This is explained by the extreme sparsity of the target: with 99.25% of FREQ values equal to zero, all models converge toward predicting near-zero frequencies for most records, making the frequency model choice less decisive than the severity model. XGBoost was nonetheless selected as the final model due to its marginally lower RMSE, native GPU acceleration enabling faster iteration, and its Poisson objective which is more theoretically appropriate for count data than the squared-error loss used implicitly by Ridge and Random Forest.

4.4 Hyperparameter Tuning

Hyperparameter tuning was conducted via a coordinate search on the XGBoost frequency model, varying one parameter at a time while holding others fixed and evaluating on a single held-out fold. The results revealed that the model is remarkably robust to hyperparameter choice on this dataset: RMSE differences across all tested values were less than 0.3 units, ranging from 7,043.5 to 7,043.8. This insensitivity is again attributable to the 99.25% zero-inflation of FREQ - the dominant signal is the structural sparsity, which all parameter configurations capture equally well. The final configuration (max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8) was retained as it represents a well-regularised, standard configuration consistent with gradient boosting best practices. Deeper trees (max_depth=8) showed no improvement and risk overfitting on the small number of non-zero training samples.

4.5 Frequency Model (FREQ)

The frequency model uses a Poisson objective (objective='count:poisson'), the canonical choice for count data. Unlike squared-error regression, the Poisson loss enforces non-negative predictions and accounts for the Poisson variance-mean relationship typical of rare event counts. The model is trained on all 383,610 observations, including the 99.25% with FREQ = 0, allowing it to learn structural factors distinguishing zero-event and non-zero-event farms.

Hyperparameter	Value	Rationale
objective	count:poisson	Appropriate for count/rate targets
n_estimators	1,200	Sufficient depth without overfitting
max_depth	6	Balanced complexity for 370 features
learning_rate	0.05	Conservative step size for stability
subsample	0.8	Row subsampling for regularisation
colsample_bytree	0.8	Feature subsampling per tree

Table 4. XGBoost hyperparameter configuration for the frequency model.

4.6 Severity Model (CM)

The severity model predicts log-transformed average claim cost, log(1 + CM), using a squared-error objective. Log-transformation is applied because CM spans several orders of magnitude (EUR 1 to EUR 500,000) with skewness of 3.83; the log scale compresses the heavy tail and stabilises training. Predictions are back-transformed via expm1(). Critically, the severity model is trained exclusively on the 2,352 records where FREQ > 0, avoiding structural zeros that would bias severity estimates downward.

Hyperparameter	Value	Rationale
objective	reg:squarederror	Regression on log-scale target
n_estimators	1,500	More trees for small training set
max_depth	8	Deeper trees for complex severity patterns
learning_rate	0.03	Slower learning compensates for small n
subsample	0.8	Stochastic row sampling
colsample_bytree	0.8	Stochastic column sampling

Table 5. XGBoost hyperparameter configuration for the severity model.

4.7 Final Premium Computation

The final predicted premium combines both model outputs with the policy duration:

$$\text{CHARGE_pred} = \max(\text{FREQ_pred} \times \text{CM_pred} \times \text{ANNEE_ASSURANCE}, 0)$$

A floor of zero is applied to prevent negative premium predictions. The cluster label derived from K-Means is included as an additional categorical feature in both models, allowing them to leverage the risk-segment information from unsupervised exploration.

5. Model Evaluation and Results

5.1 Evaluation Methodology

Model performance is measured using Root Mean Squared Error (RMSE) on the CHARGE variable, consistent with Credit Agricole's evaluation criterion. RMSE penalises large errors quadratically, appropriate in an insurance context where extreme mispricing of high-risk policies is particularly costly. We employ 5-fold cross-validation (KFold, shuffle=True, random_state=42) to obtain stable out-of-sample error estimates.

5.2 Cross-Validation Results

Fold	Train Rows	Val Rows	RMSE (EUR)
1	306,888	76,722	7,044.76
2	306,888	76,722	6,984.65
3	306,888	76,722	7,156.34
4	306,888	76,722	6,333.84
5	306,888	76,722	6,431.90
Mean +/- Std	-	-	6,790.30 +/- 338.53

Table 6. 5-fold cross-validation RMSE results. [Figure 5 - CV Bar Chart]

5.3 Business Context and Model Trade-offs

Beyond raw predictive performance, the model is assessed against broader business criteria:

Criterion	Assessment	Rating
Predictive accuracy	CV RMSE 6,790; LB RMSE 5,603	****
Interpretability	Feature importances available; cluster profiles readable	***
Regulatory compliance	No protected attributes; some black-box risk	***
Computational cost	~15 min GPU training; lightweight inference	****
Stability	Low CV variance (std = 338, ~5% of mean)	*****
Calibration	Poisson FREQ + log-normal CM; actuarially grounded	****
Maintainability	Single Python script, retrainable on new data	****

Table 7. Business trade-off evaluation of the final model.

5.4 Competition Result

The model achieved a leaderboard RMSE of **5,603.03** on the course competition. The best competing submission scored 5,594.18, placing our model within the top results. The gap between cross-validation RMSE (6,790) and leaderboard RMSE (5,603) indicates the model generalises well to the held-out test distribution, with the CV estimate being conservative due to the small number of non-zero CM training samples (~470 per fold) inflating fold-level variance.

6. Deployment at Credit Agricole Assurances

A key requirement for production deployment is integration into Credit Agricole's existing IT infrastructure. We propose a microservice architecture consisting of four components.

6.1 Model Serialisation and Versioning

Both XGBoost models (FREQ and CM) are serialised using the native XGBoost binary format (model.save_model()) or Python joblib interface. Model versioning is managed through MLflow or a similar experiment-tracking platform, ensuring reproducibility and enabling rollback to prior versions when retrained models underperform on recent data.

6.2 REST API Inference Service

The serialised models are wrapped in a lightweight REST API (FastAPI or Flask) deployed as a containerised service (Docker). The API accepts a JSON payload of 370 raw feature values for a new policyholder, applies the same preprocessing pipeline (imputation, ordinal encoding, cluster assignment), calls both models in sequence, and returns predicted FREQ, CM, and CHARGE. The entire inference pipeline executes in under 100ms per request, suitable for real-time underwriting workflows.

6.3 Batch Scoring for Portfolio Repricing

For annual portfolio repricing, a batch scoring pipeline is scheduled as a nightly job. The pipeline reads updated exposure data from the core insurance system, applies the models, and writes predicted premiums back to the pricing database. This decouples model inference from the underwriting UI and avoids recomputation at query time.

6.4 Monitoring and Retraining

Model performance is monitored in production by comparing predicted premiums against observed claims at annual intervals. Population Stability Index (PSI) checks detect feature distribution drift. When RMSE on recent data exceeds a defined threshold or PSI flags distributional shift, the models are automatically retrained on the updated dataset, ensuring the system adapts to changing agricultural risk patterns.

Component	Technology	Purpose
Model storage	XGBoost binary / MLflow	Versioning and reproducibility
Inference API	FastAPI + Docker	Real-time premium quotation
Batch pipeline	Apache Airflow / Spark	Portfolio repricing
Feature store	PostgreSQL / Redis	Consistent feature serving
Monitoring	Evidently AI / custom PSI	Drift detection and alerting
Retraining trigger	Scheduled + threshold	Model freshness

Table 8. Proposed deployment architecture components.

7. Conclusion

This report has presented a complete end-to-end machine learning pipeline for predicting fire insurance premiums for agricultural firms. The key contributions are:

- **Unsupervised insight:** Unsupervised insight: PCA and K-Means revealed five distinct risk segments, with Cluster 2 showing a mean premium 7x higher than the lowest-risk segment.
- **Actuarially grounded modelling:** Actuarially grounded modelling: The frequency-severity decomposition with Poisson FREQ and log-normal CM reflects established actuarial practice.
- **Competitive performance:** Competitive performance: A leaderboard RMSE of 5,603 places the model within best submissions.
- **Production readiness:** Production readiness: The proposed microservice deployment architecture enables seamless integration into Credit Agricole's IT systems.

Future work could explore LightGBM or CatBoost as alternative gradient boosting implementations, ensemble stacking of multiple algorithms, and explicit modelling of the zero-inflation structure via a hurdle model. Incorporating temporal features (year-on-year claim history) and external data sources (satellite-based fire risk indices, crop calendars) could further improve severity predictions for challenging long-tail cases.

Appendix A: Key Code Excerpts

A.1 Feature Classification

```
binary_cols = [col for col in x_train.columns
    if set(x_train[col].dropna().unique()).issubset({0,1,0.0,1.0})]
cat_cols = list(set(raw_cat + x_train.select_dtypes('object').columns.tolist()))
numerical_cols = [col for col in x_train.columns
    if col not in cat_cols and col not in binary_cols]
```

A.2 Frequency Model Training

```
freq_model = XGBRegressor(
    objective='count:poisson', n_estimators=1200,
    max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8, random_state=42)
freq_model.fit(X_train_split, y_freq_tr)
freq_pred = freq_model.predict(X_val)
```

A.3 Severity Model Training

```
mask = y_freq_tr > 0       # train only on non-zero frequency rows
y_cm_log = np.log1p(y_cm_tr[mask])
cm_model = XGBRegressor(
    objective='reg:squarederror', n_estimators=1500,
    max_depth=8, learning_rate=0.03,
    subsample=0.8, colsample_bytree=0.8, random_state=42)
cm_model.fit(X_train_split[mask], y_cm_log)
cm_pred = np.maximum(np.expm1(cm_model.predict(X_val)), 0)
```

A.4 Final Premium Computation

```
# Final premium: frequency x severity x policy duration
final_pred = freq_pred * cm_pred * X_val['ANNEE_ASSURANCE']
rmse = np.sqrt(mean_squared_error(y_charge_val, final_pred))
```

A.5 Top Feature Importances - Frequency Model

Rank	Feature	Domain Interpretation
1	NBBAT2	Number of buildings (type 2) - exposure proxy
2	NBBAT10	Number of buildings (type 10) - exposure proxy
3	ANNEE_ASSURANCE	Policy duration - scales total exposure
4	RISK1	Risk classification score
5	Cluster	K-Means segment - risk stratification
6	SURFACE19	Agricultural surface area type 19
7	BDTOPO_BAT_MAX_HAUTEUR	Maximum building height (topographic data)
8	KAPITAL32	Insured capital category 32

Table A.1. Top 8 features by XGBoost gain importance for the frequency model.